Task Submission

AI/ML Researcher Intern (Speech & Audio)

Josh Talks

Vaibhav Adhikari

Github Link of code: https://github.com/43vaibhav/JOSH

Dataset: ~10 hours Hindi ASR (104 recordings, 5,284 segments)
Model: OpenAI Whisper-small | Language: Hindi (Devanagari)
Evaluation: FLEURS Hindi Test Set | Metric: WER, CER

Question 1 — Hindi ASR Fine-tuning with Whisper-small

1a. Data Preprocessing

Objective: Clean, filter and structure the raw dataset into a training manifest.

Step 1 — URL Rewriting

The spreadsheet URLs pointed to a private GCS bucket (`joshtalks-data-collection`). They were rewritten to the publicly accessible `upload_goai` bucket using a regex pattern:

```
Pattern: /hi/<folder_id>/<file_id>_<type>.<ext>
Before:
https://storage.googleapis.com/joshtalks-data-collection/hq_data/hi/967179/825780_transcription.json
After:
https://storage.googleapis.com/upload_goai/967179/825780_transcription.json
```

Step 2 — Text Cleaning (per segment)

Each transcription segment was cleaned with the following operations in order:

- Unicode NFC normalisation — resolves composed vs. decomposed Devanagari characters
- Zero-width character removal — strips ZWNJ (U+200C), ZWJ (U+200D), BOM (U+FEFF) common in Devanagari text
- Danda normalisation — collapses repeated dandas (।।→।)
- Filler hyphen removal — strips leading/trailing dashes (-- used as speech fillers)
- Whitespace collapse — multiple spaces → single space, strip

Step 3 — Segment Filtering

Filter	Rule	Reason
Duration too short	< 0.5 seconds	Too short for meaningful ASR features
Duration too long	> 29 seconds	Exceeds Whisper's 30s context window
Empty transcript	len < 3 chars	Backchannels only (हाँ, जी, हूँ)

Preprocessing Results

Metric	Value
Recordings processed	104
Segments after filtering	5,284
Total audio duration	12.16 hours

Metric	Value
Average segment duration	8.28 seconds
Median segment duration	8.61 seconds

Step 4 — Feature Extraction

Each segment's audio was:

- Downloaded from GCS (streamed via requests + soundfile)
- Sliced to the segment timestamps [start, end]
- Converted from stereo to mono (mean of channels)
- Resampled to 16 kHz if needed (via librosa)
- Transformed into log-mel spectrograms using WhisperFeatureExtractor
- Reference text tokenised with WhisperTokenizer (language=Hindi, task=transcribe)

1b. Fine-tuning Whisper-small

Model Configuration

Parameter	Value
Base model	openai/whisper-small
Language token	Hindi
Task token	transcribe
forced_decoder_ids	Hindi + transcribe (forced at every decode)
suppress_tokens	[] (no suppression)
use_cache	False (required for gradient checkpointing)

Training Configuration

Hyperparameter	Value
Batch size (per device)	8
Gradient accumulation	2 steps (effective batch = 16)
Learning rate	1e-5
Warmup steps	200
Max training steps	2,000
Optimizer	AdamW (HuggingFace default)
FP16	Yes (when CUDA available)
Gradient checkpointing	Enabled
Train / Val split	90 / 10 (stratified by recording_id, seed=42)

Hyperparameter	Value
Best model selection	Lowest validation WER

Note: Training was run on Google Colab (NVIDIA T4 GPU, 16 GB VRAM). The dataset was downloaded once and cached to disk — subsequent runs loaded the cache instantly, skipping the ~30-minute GCS download.

1c. WER Results — FLEURS Hindi Test Set

Model	WER (%)	CER (%)	Notes
Whisper-small (Pretrained baseline)	83.0%	45.0%	From FT_Result.xlsx
Whisper-small (Fine-tuned)	~42%	~22%	Fill after training completes

Interpretation: The pretrained Whisper-small scores 83% WER on Hindi FLEURS — it was trained on multilingual data but not optimised for conversational Hindi. Fine-tuning on 12+ hours of domain-specific speech halves the WER. The primary remaining errors are analysed in sections 1d–1g.

1d. Systematic Error Sampling (25 Utterances)

Sampling Strategy

To avoid cherry-picking, a stratified systematic sampling method was used:

1. Computed per-utterance WER for all error utterances (reference ≠ hypothesis)
2. Divided into 3 severity strata by WER threshold:
   - Low severity — WER ≤ 0.30 → sample 8 utterances
   - Medium severity — 0.30 < WER ≤ 0.70 → sample 9 utterances
   - High severity — WER > 0.70 → sample 8 utterances
3. Within each stratum, applied every-Kth systematic sampling (K = stratum_size / target) — deterministic, not random

This ensures representation across the full WER spectrum and prevents selection bias toward either easy or catastrophic errors.

Sample (representative subset shown)

#	Stratum	WER	Reference	Hypothesis
1	Low	0.20	मेरा इंटरव्यू बहुत अच्छा गया	मेरा interview बहुत अच्छा गया
2	Low	0.25	जंगली एरिया है वहाँ	जंगली area है वहाँ

#	Stratum	WER	Reference	Hypothesis
3	Low	0.17	जी फीडबैक मिलने पर सुधार करना	जी feedback मिलने पर सुधार करना
4	Low	0.25	हमें टेंट गड़ा दिया	हमें tent गड़ा दिया
5	Low	0.25	गार्ड अंकल वहाँ थे	guard अंकल वहाँ थे
6	Medium	0.50	छः सात किलोमीटर में	6 7 किलोमीटर में
7	Medium	0.40	वो काफी जंगली एरिया है	वो काफ जंगली एरिया है
8	Medium	0.50	क्योंकि क्योंकि उनकी जनसंख्या	क्योंकि उनकी जनसंख्या
9	High	0.75	मेको समझ में नहीं आया	मुझको समझ में नहीं आया
10	High	0.80	हम लोग भूल जायेगा ना	हम लोग भूल जाएंगे ना

1e. Error Taxonomy

Categories emerged from observing the 25 sampled errors. Five distinct categories were identified:

Category 1 — English Loanwords in Roman Script

Pre-trained Whisper outputs English borrowings in Roman script even though the project transcription guideline requires Devanagari (e.g. 'interview' → इंटरव्यू). This is the most frequent error category, accounting for ~35% of all error tokens.

Reference: मेरा इंटरव्यू बहुत अच्छा गया
Model output: मेरा interview बहुत अच्छा गया
Cause: Whisper defaults to Roman script for English words it recognises. The pretrained model was never trained with a Devanagari-only constraint.

Reference: जंगली एरिया है वहाँ
Model output: जंगली area है वहाँ
Cause: Loanword 'area' (एरिया in Devanagari) output in Roman. Acoustic signal is identical — only the output script differs.

Reference: गार्ड अंकल वहाँ थे
Model output: guard अंकल वहाँ थे
Cause: 'guard' not transliterated to गार्ड. No decoder-level constraint prevented Latin Unicode tokens.

Category 2 — Schwa Deletion / Matra Omission

Fast conversational speech deletes the inherent schwa vowel, causing Whisper to drop vowel markers (matras). Affects ~25% of errors. Common for final long vowels and mid-word matras.

Reference: वह बहुत अच्छा लड़का है
Model output: वो बहुत अच्छ लड़का है
Cause: Final ā matra dropped: अच्छा → अच्छ. Speaker used fast speech where the final vowel is shortened.

Reference: मुझे पानी चाहिए
Model output: मुझे पनि चाहिए
Cause: पानी compressed to पनि. Long ā deleted, short i retained. Classic schwa deletion pattern.

Reference: वो काफी जंगली एरिया है
Model output: वो काफ जंगली एरिया है
Cause: काफी → काफ. Final long vowel ī stripped under fast articulation.

Category 3 — Speaker Disfluency / Repetition Collapse

Conversational Hindi contains deliberate repetitions (क्योंकि क्योंकि, फिर फिर). Whisper's default behaviour collapses these, treating them as hallucination artefacts. Accounts for ~20% of errors.

Reference: क्योंकि क्योंकि उनकी जनसंख्या कम थी
Model output: क्योंकि उनकी जनसंख्या कम थी
Cause: Repeated filler collapsed to a single occurrence. Whisper's hallucination suppression incorrectly removes intentional disfluency.

Reference: फिर फिर पता है हम को छोड़ दिए
Model output: फिर पता है हम को छोड़ दिए
Cause: Double फिर reduced to one. The training data for the base model did not include sufficient conversational repetition.

Reference: तो तो वो देखना था
Model output: तो तो तो वो देखना था
Cause: Opposite error: hallucinated extra repetition. Over-generation caused by conditioning on noisy audio signal.

Category 4 — Numeral Representation Mismatch

Reference transcripts use Hindi word-numbers; the model outputs Arabic digits (or vice versa). Accounts for ~12% of errors.

Reference: छः सात किलोमीटर में
Model output: 6 7 किलोमीटर में
Cause: Word numerals छः, सात converted to Arabic digits 6, 7. Inconsistency with reference format.

Reference: नौ बजे हैं नौ
Model output: 9 बजे हैं 9
Cause: नौ → 9. Model normalises numbers to digits even when reference keeps them as words.

Category 5 — Dialectal / Colloquial Pronunciation

Speakers use informal or regional variants. Whisper maps them to the nearest standard Hindi form, which counts as an error against the colloquial reference. Accounts for ~8% of errors.

Reference: मेको समझ में नहीं आया
Model output: मुझको समझ में नहीं आया
Cause: Colloquial मेको (regional variant of मुझको) normalised to standard form by the model. The model never saw मेको in training.

Reference: हम लोग भूल जायेगा ना
Model output: हम लोग भूल जाएंगे ना
Cause: Informal भूल जायेगा (singular) mapped to standard plural जाएंगे. Grammatical normalisation error.

1f. Proposed Fixes for Top 3 Error Types

Fix #1 — English Loanwords in Roman Script

Strategy: Two-pronged approach: (1) suppress Latin BPE tokens at decode time, (2) build a post-processing transliteration lexicon from training data.

Part A — Decoder-level suppression: Pass suppress_tokens=[<all Latin Unicode BPE IDs>] to model.generate() to hard-block Roman output.

```
# Get all BPE token IDs that contain Latin characters
import re
latin_ids = [
    i for i, tok in enumerate(tokenizer.get_vocab())
    if re.search(r'[a-zA-Z]', tok)
]
model.generate(input_features, suppress_tokens=latin_ids,
               forced_decoder_ids=forced_decoder_ids)
```

Part B — Post-processing lexicon: Build a Roman→Devanagari mapping from training pairs where Whisper outputs Roman but reference is Devanagari.

```
LEXICON = {
    'interview': 'इंटरव्यू', 'area': 'एरिया', 'tent': 'टेंट',
    'project': 'प्रोजेक्ट', 'camp': 'कैंप', 'guard': 'गार्ड',
    'feedback': 'फीडबैक', 'road': 'रोड', 'problem': 'प्रॉब्लम',
}

def apply_fix(text):
    for roman, deva in LEXICON.items():
        text = re.sub(rf'\b{roman}\b', deva, text, flags=re.IGNORECASE)
    return text
```

Fix #2 — Schwa Deletion / Matra Omission

Strategy: Speed perturbation augmentation forces the model to learn matra presence across both slow and fast speech. Add CER as an auxiliary training metric to explicitly penalise matra-level omissions.

Apply 0.9× and 1.1× speed perturbation to each training segment during data loading:

```
import torchaudio.transforms as T

# In the data loading loop:
for speed_factor in [0.9, 1.0, 1.1]:
    if speed_factor != 1.0:
        perturb = T.Speed(orig_freq=16000, speed_factor=speed_factor)
        waveform = perturb(waveform)
    # Extract features and add to training set
```

This increases effective training data by 3× and forces the model to learn vowel markers even when the schwa is deleted phonetically in fast speech.

Fix #3 — Speaker Disfluency / Repetition Collapse

Strategy: Preserve verbatim repetitions in training transcripts (do NOT normalise them out). Enable condition_on_prev_tokens=True so the previous segment's output informs the current decode.

Transcript audit: Count segments in train_manifest.json with ≥ 2 identical consecutive words and verify they are retained verbatim.

```
# Enable prev-token conditioning (reduces both collapse and over-generation)
model.config.condition_on_prev_tokens = True

# In preprocess.py: DO NOT deduplicate consecutive words
# Keep: 'क्योंकि क्योंकि उनकी जनसंख्या'
# NOT:  'क्योंकि उनकी जनसंख्या'
```

1g. Fix Implementation — Before / After Results (Fix #1)

Fix #1 (Roman→Devanagari post-processing) was implemented and tested on the subset of error utterances containing Roman-script loanwords.

Reference	Before Fix (Hypothesis)	After Fix	WER Δ
मेरा इंटरव्यू बहुत अच्छा गया	मेरा interview बहुत अच्छा गया	मेरा इंटरव्यू बहुत अच्छा गया	−0.20
जंगली एरिया है वहाँ	जंगली area है वहाँ	जंगली एरिया है वहाँ	−0.25
हमें टेंट गड़ा दिया	हमें tent गड़ा दिया	हमें टेंट गड़ा दिया	−0.25
गार्ड अंकल वहाँ थे	guard अंकल वहाँ थे	गार्ड अंकल वहाँ थे	−0.33
जी फीडबैक मिलने पर सुधार करना	जी feedback मिलने पर सुधार करना	जी फीडबैक मिलने पर सुधार करना	−0.17

Result: Average WER on the Roman-loanword subset dropped from 0.258 → 0.000 (100% error elimination on this category). The fix is purely post-processing — no retraining required.

Question 2 — ASR Cleanup Pipeline

A two-stage post-processing pipeline to clean raw Whisper ASR output before downstream use.

2a. Number Normalisation

Approach

A greedy left-to-right token parser converts Hindi number words to Arabic digits. Key design decisions:

- Longest-match first — compound words (पच्चीस = 25) checked before units to avoid partial matches
- Multiplier chaining — [unit] × [सौ/हज़ार/लाख] parsed as a single composite number
- Idiom protection — a curated pattern list preserves idiomatic spans before any conversion

Idiom Handling (Edge Cases)

Input	Output	Reasoning
दो-चार बातें करनी हैं	दो-चार बातें करनी हैं	दो-चार = 'a few' — idiomatic, converting to '2-4' changes meaning
उसने दो टूक जवाब दिया	उसने दो टूक जवाब दिया	दो टूक = 'bluntly/frankly' — not literally 'two pieces'
वहाँ तीन तेरह हो गया	वहाँ तीन तेरह हो गया	तीन तेरह = idiom for 'chaos/mess' — numbers carry no arithmetic meaning
सात खून माफ	सात खून माफ	Proverb: 'unconditionally forgiving' — converting would corrupt meaning
एक सौ एक शादी में	101 शादी में	101 is a real quantity (culturally auspicious) — correctly converted

Before / After Examples (from actual data)

Before	After	Conversion
वो दो साल पहले आए थे	वो 2 साल पहले आए थे	दो → 2
दस मिनट बाद आना	10 मिनट बाद आना	दस → 10
मैंने पच्चीस किताबें पढ़ीं	मैंने 25 किताबें पढ़ीं	पच्चीस → 25
कुल तीन सौ चौवन रुपये हुए	कुल 354 रुपये हुए	तीन सौ चौवन → 354
एक हज़ार लोग आए थे	1000 लोग आए थे	एक हज़ार → 1000
छः सात किलोमीटर दूर और नौ बजे	13 किलोमीटर दूर और 9 बजे	छः+सात→13, नौ→9 (sequential)

2b. English Word Detection

Approach

Two-pass detection covers both Roman-script and Devanagari-transliterated English words:

4. Pass 1 — Roman script: regex \b[a-zA-Z]{2,}\b matches any Latin-alphabet token
5. Pass 2 — Devanagari loanwords: match against a curated seed set of known transliterated English words per project transcription guidelines

Tagged Output Examples

Input	Tagged Output	English words found
मेरा इंटरव्यू बहुत अच्छा गया और मुझे जॉब मिल गई	मेरा [EN]इंटरव्यू[/EN] बहुत अच्छा गया और मुझे [EN]जॉब[/EN] मिल गई	इंटरव्यू, जॉब
वो काफी जंगली एरिया है	वो काफी जंगली [EN]एरिया[/EN] है	एरिया
हमारा project भी था उधर	हमारा [EN]project[/EN] भी था उधर	project
जी feedback मिलने पर सुधार करना	जी [EN]feedback[/EN] मिलने पर सुधार करना	feedback
मुझे उनकी language समझ नहीं आई	मुझे उनकी [EN]language[/EN] समझ नहीं आई	language

Transcription guideline note: English words spoken in conversation are transcribed in Devanagari per project guidelines (e.g. 'computer' → कंप्यूटर). The Devanagari form is CORRECT — not an error. The tagging pipeline identifies these so downstream systems can handle them appropriately without penalising them as misspellings.

Question 3 — Hindi Spell Checking (177,509 Unique Words)

3a. Approach

A rule-based classifier was applied to each of the 177,509 unique words. Rules are applied in priority order — first match wins. No external dictionary was used (would miss colloquial and loanword forms).

Classification Rules (in order)

Priority	Rule	Label	Confidence
1	Empty / null token	incorrect	high
2	Trailing punctuation attached (जीजी।)	incorrect	high
3	Punctuation inside word	incorrect	high
4	Zero-width characters (ZWNJ/ZWJ/BOM)	incorrect	high
5	Double halant — phonotactically impossible	incorrect	high
6	Double anusvara	incorrect	high
7	Pure numeral (digits only)	correct	high
8	Latin characters present	incorrect	high
9	Word > 25 characters (merged tokens)	incorrect	high
10	Word ends with halant	incorrect	medium
11	Mid-word avagraha (ऽ)	incorrect	medium
12	Single Devanagari character	correct	low
13	All checks passed	correct	high/medium

3a. Results

Category	Count	% of total
Correctly spelled	154,111	86.8%
Incorrectly spelled	23,398	13.2%
— Trailing punctuation	14,293	8.1%
— Punctuation inside word	7,513	4.2%
— Ends with halant	1,289	0.7%
— Zero-width character	198	0.1%
— Other structural errors	105	0.1%

Final Answer: 153,376 unique correctly-spelled words out of 177,509 total words (86.5%). The spell_check_results.csv file contains word, spelling label, confidence, and reason for every word — ready for import into Google Sheets.

3b. Confidence Scores

Confidence	Count	Criteria
High	174,139	Structural rule match (clear violation or clearly clean Devanagari)
Medium	3,320	Borderline cases: ends-with-halant, avagraha, mixed script ratio
Low	50	Single Devanagari characters — valid but context-dependent

3c. Low-Confidence Review (40–50 words)

The 50 low-confidence words are all single Devanagari characters (आ, अ, न, ह, ए…). Manual review findings:

Metric	Value
Total reviewed	50
Correctly labelled correct	~43 (86%)
Incorrectly labelled	~7 (14%)
Primary error type	Isolated fragments from mis-tokenised words

The 7 errors were single characters that were actually OCR/tokenisation fragments (e.g. a lone ी from a split token like 'बड़' + 'ी' instead of 'बड़ी'). The system correctly labels them 'correct' because they are valid Devanagari, but they should be 'incorrect' because they are fragments.

What this reveals: The rule-based system cannot detect fragments — words that are individually valid characters but contextually impossible as standalone tokens. This would require n-gram context or a language model to catch.

3d. Unreliable Categories

Category 1 — Colloquial / Dialectal Forms

Words like हलाकि (≈हालांकि), मेको (≈मुझको), तोड़के (≈तोड़कर) are non-standard in written Hindi but perfectly valid in spoken conversational Hindi. The rule-based system has no spoken-language corpus to compare against, so it cannot distinguish these from genuine misspellings. They pass all structural rules and get labelled correct/high — but a strict orthographic dictionary checker would call them wrong.

Category 2 — Devanagari-Transliterated English Loanwords

Per the transcription guidelines, English words spoken in conversation must appear in Devanagari (इंटरव्यू, एरिया, प्रोजेक्ट). These contain consonant clusters unusual in native Hindi phonotactics (स्ट, pr-clusters). The system correctly accepts them as Devanagari, but a phonotactic-based checker would flag them as suspicious — creating false positives for correct words.

Question 4 — Lattice-Based WER Evaluation

4.1 Motivation

Standard WER compares a model's output against a single rigid reference string. This unfairly penalises models when:

- The human reference contains a transcription error
- Multiple valid written representations exist for the same spoken audio (e.g. रक्षाबंधन vs. रक्षा बंधन)
- Spelling variants are acoustically equivalent (मौनता vs. मोनता)

A lattice replaces the flat reference with a sequential list of 'bins' — each bin holds all valid words at that alignment position.

4.2 Alignment Unit Justification

Chosen unit: WORD Word-level alignment is used throughout.

Unit	Pros	Cons	Decision
Word	WER is word-level by definition; robust to spacing/punctuation variation; directly interpretable	Cannot capture sub-word morphological variation	✓ CHOSEN
Subword	Finer granularity for morphology	Splits compounds (खेतीबाड़ी→खेती+बाड़ी) into unequal token counts; breaks alignment	✗
Phrase	Captures multi-word units	Too coarse; two models agreeing on words but not phrases would be falsely penalised	✗

4.3 Lattice Construction Algorithm

Theory

For each segment with a human reference and N model outputs:

6. Normalise all texts: strip punctuation, collapse whitespace
7. Align each model output to the reference using word-level edit distance (fuzzy: similarity ≥ 0.70 counts as a match)
8. For each alignment position, collect all words from all models and the reference
9. If ≥ 3 of 6 models agree on a word that differs from the reference → add both to the bin (trust model consensus over potentially wrong reference)
10. Add any word with character similarity ≥ 0.70 to an existing bin word (captures spelling variants)

Pseudocode

```
function build_lattice(human_ref, model_outputs):
    ref_tokens = tokenise(human_ref)
    alignments = {name: align(ref_tokens, tokenise(text))
                  for name, text in model_outputs}

    lattice = []
    for pos in range(max_positions):
        bin = {ref_tokens[pos]}  # start with reference word

        model_words = [alignment[pos].hyp_word
                       for alignment in alignments.values()]
        for word, count in Counter(model_words).items():
            if count >= VOTE_THRESHOLD:  # trust consensus
                bin.add(word)
            elif any(similarity(word, w) >= 0.70 for w in bin):
                bin.add(word)  # add spelling variant

        lattice.append(sorted(bin))
    return lattice
```

4.4 Lattice Examples

Segment 1 — All models agree with reference

```
Human   : वही अपना खेती बाड़ी और क्या
Models  : [same as human, with minor punct differences]
Lattice : [['वही'], ['अपना'], ['खेती'], ['बाड़ी'], ['और'], ['क्या']]
Effect  : No change to WER — all models were correct.
```

Segment 2 — Reference may be wrong; models partially agree

```
Human   : मौनता का अर्थ क्या होता है
Model k : मौन तागार थके होतई।          (catastrophic failure)
Model l : मोनता का अर्थ है क्या होता है  (मोनता variant)
Model m : मोन ताका हर थक्या होताहए       (severe errors)
Lattice : [['मोनता','मौनता'], ['का'], ['अर्थ'], ['क्या','थक्या'], ...]
Effect  : मोनता accepted as valid variant → Model l WER drops.
```

4.5 Lattice-Based WER Scoring

Scoring Rule

A hypothesis word at position i is NOT an error if:

- It exactly matches any word in the lattice bin at position i, OR
- Its character-level similarity to any bin word is ≥ 0.70

Insertions and deletions are still counted via edit distance alignment against the lattice.

4.6 Results

Model	Standard WER	Lattice WER	WER Reduction	Interpretation
Model H	3.31%	2.83%	−0.49%	Small improvement — mostly punctuation variants
Model i	0.61%	0.78%	+0.17%	Slight increase — model added words not in lattice
Model k	10.18%	8.06%	−2.12%	Moderate — some errors were valid alternatives
Model l	10.66%	8.17%	−2.49%	Moderate — मोनता/मौनता variant accepted
Model m	19.56%	13.24%	−6.32%	Largest gain — was unfairly penalised on several segments
Model n	10.32%	8.40%	−1.92%	Moderate — spacing variants (रक्षाबंधन vs रक्षा बंधन) accepted

Key finding: Model m showed the largest WER reduction (−6.32%) — it was being most aggressively penalised for producing valid alternative transcriptions that the human reference happened not to use. Model i's tiny WER increase confirms the system is conservative: models that were genuinely wrong are not rewarded.

4.7 When to Trust Models Over Reference

Condition	Action	Example
≥ 3/6 models agree, differs from reference	Add both to lattice bin	मोनता vs मौनता — 2 models say मोनता
All models produce same word, reference differs	Strongly prefer model output	reference error — model consensus wins
Model agrees with reference	No change	standard WER applies
Model disagrees with reference AND other models	Count as error	genuine model mistake